Technische Universität Braunschweig

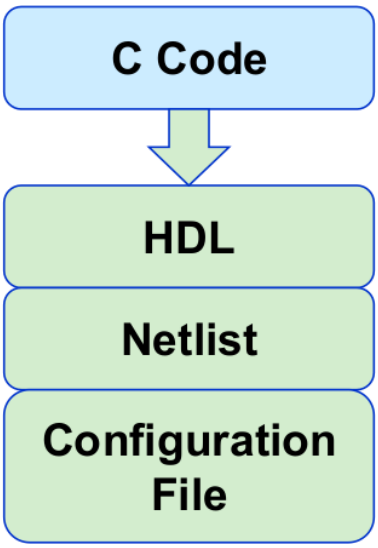
Chair for Chip Design for Embedded Computing

Hardware-Software-Systeme

Kapitel 08 - High-Level Synthese

Prof. Dr.-Ing. Guillermo Payá Vayá, 28. Januar 2025

High-Level Synthese

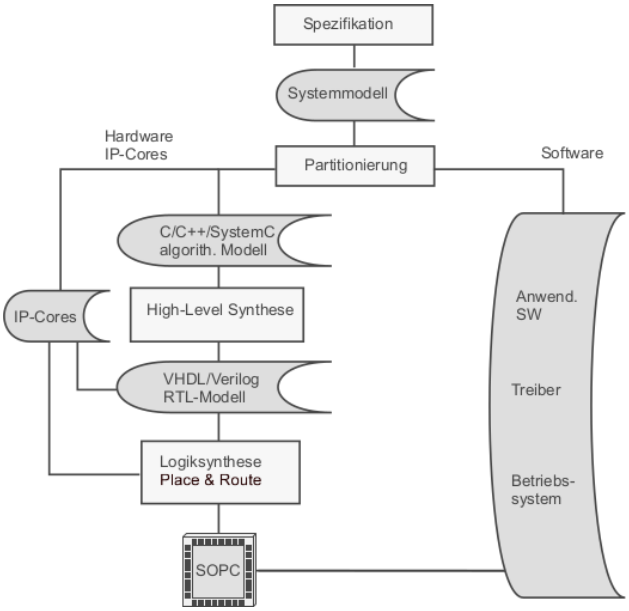


- Trend: **High-Level Synthese**
  - Vereinfachter Entwicklungsprozess
  - Einheitliche Verifikation
  - Vereinfachter Einstieg in HW-Entwicklung
- Was bedeutet High-Level Synthese
  - CAD Tools: Höhere Abstraktion
    → Funktionalitätsebene, kein Timing
  - Neue Sprache: Optimal, aber Einarbeitung erforderlich
  - Weiterentwicklung traditioneller oder Objekt-orientierter Sprachen

→ Übersetzung in HW durch einen Compiler
→ Konzentration des Programmierers auf den Algorithmus

Rolle im Entwurf von SoPC

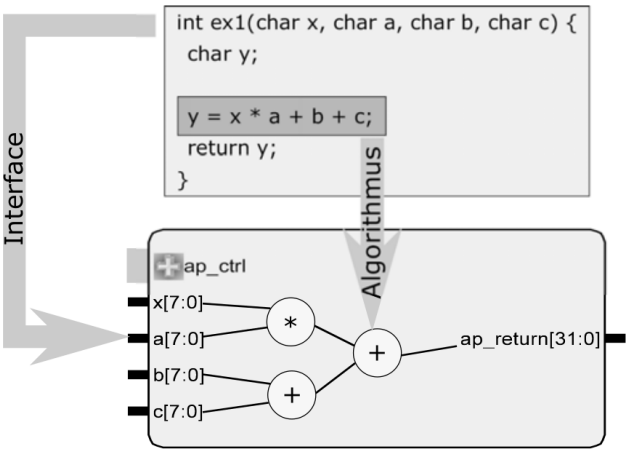
- Moderne SoPCs vereinen auf einer integrierten Schaltung leistungsfähige Prozessorsysteme mit einer programmierbaren Logik (FPGA) und werden daher auch als hybride CPU-FPGA-Systeme bezeichnet.
  - → Ein Beispiel hierfür ist der *Zynq-SoPC* von Xilinx, welcher zwei ARM Cortex-A9-CPUs mit einem Artix/Kintex-7 FPGA auf dem gleichen IC integriert



Interface-Synthese

Interface- und Algorithmen-Synthese (I)

- Die Aufgabe der HLS ist es, ein in C oder C++ (oder SystemC) geschriebenes Programm in eine Hardwareimplementierung umzusetzen.
- Bei der Synthese muss einerseits die Funktionalität oder der ***Algorithmus*** durch digitale Hardware implementiert werden und andererseits muss auch das ***Interface*** der Schaltung, das sind die Input- und Output-Ports, implementiert werden.



Interface-Synthese

- Die Interface- und die Algorithmus-Synthese können durch Direktiven vom Entwickler gesteuert werden.
- Die Funktionsargumente und der Return-Wert der Toplevel-Funktion definieren das Interface.
  - → Dabei spielt der Datentyp und die Art der Argumente eine Rolle, beispielsweise ob ein Array als Argument übergeben wird.
- Vom Entwickler kann dann beispielsweise spezifiziert werden, ob für die *Interface-Ports spezielle Protokolle* zu implementieren sind, von *einfachen Handshake-Protokollen* bis hin zu *Busprotokollen für AXI-Master- oder AXI-Slave-Schnittstellen*, welche eine Einbettung in einem SoPC wie dem Zynq ermöglichen.

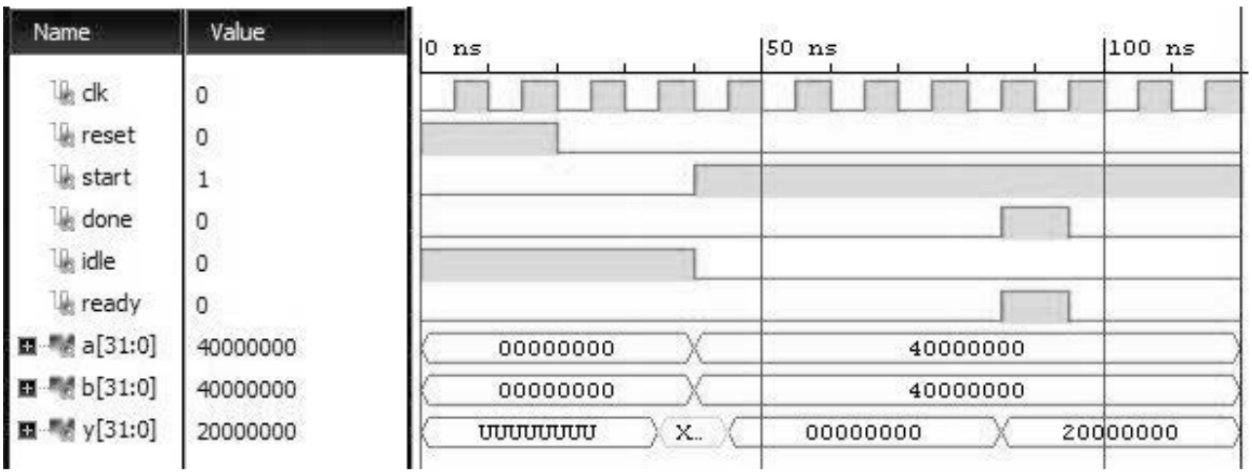
(Default) Block-Protokol (I)

- Das **Block-Level**-Protokoll steuert die Ausführung einer Operation des ganzen IP Cores über ein *Handshake*-Protokoll und ist unabhängig von **Port-Level**-Protokollen für die einzelnen Daten-Ports.
- Unter der Operation versteht man dabei die Ausführung der Funktion, dies entspricht im C/C ++ -Programm dem Aufruf dieser Funktion.

```c
#include "func3.h"

out_t func3(in_t a, in_t b) {
  return (out_t) a * b;
}
```

```vhdl
entity func3 is
  port (
    ap_clk    : IN  STD_LOGIC;
    ap_rst    : IN  STD_LOGIC;
    ap_start  : IN  STD_LOGIC;
    ap_done   : OUT STD_LOGIC;
    ap_idle   : OUT STD_LOGIC;
    ap_ready  : OUT STD_LOGIC;
    a_V       : IN  STD_LOGIC_VECTOR (31 downto 0);
    b_V       : IN  STD_LOGIC_VECTOR (31 downto 0);
    ap_return : OUT STD_LOGIC_VECTOR (31 downto 0));
end;
```

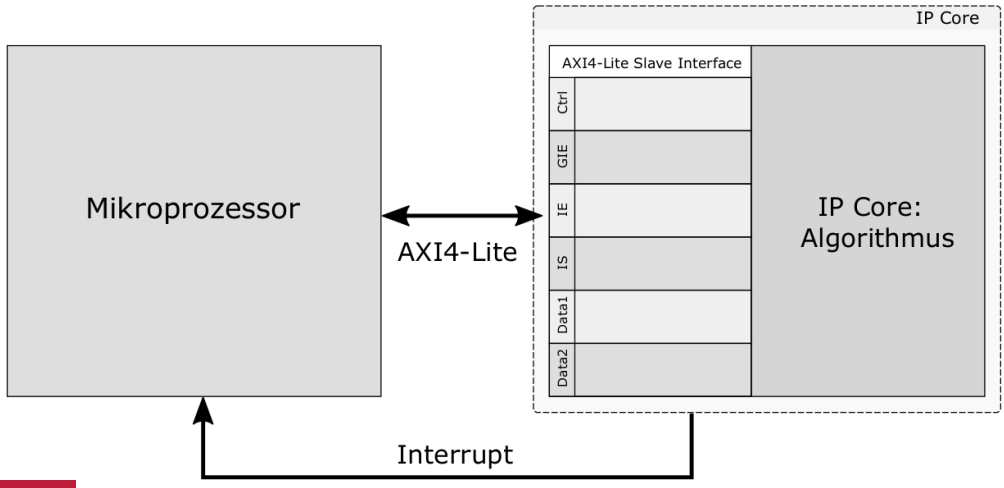

(Default) Block-Protokoll (II)



- **ap_start** (Input): Startet die Abarbeitung einer Operation des IP Cores.
- **ap_done** (Output): Zeigt bei aktivem Pegel an, dass die Abarbeitung einer Operation beendet ist und der Rückgabewert der Funktion gültig ist.
- **ap_idle** (Output): Zeigt bei aktivem Pegel an, dass der IP Core keine Operation durchführt.
- **ap_ready** (Output): Zeigt bei aktivem Pegel an, dass neue Eingangswerte angelegt werden können.

AXI4-Interface (I)

- Wie aus der Mikroprozessor- und Mikrocontroller-Technik bekannt ist, werden Peripherieeinheiten in der Regel als *Memory-Mapped-I/O* in das System eingebunden.
- Dies ist auch bei den Xilinx-Systemen so und wird durch das ***AMBA/AXI4-Bussystem*** unterstützt



AXI4-Interface (II)

```
int ex5(int a[5]) {
#pragma HLS INTERFACE s_axilite port=return
#pragma HLS INTERFACE s_axilite port=a

  int sum = 0;
  for(int i=0; i<5; i++){
    sum += a[i];
  }
  return sum;
}
```

Adress-Offset	Bit	Funktion	
0x00		Control	
	Bit 0	ap_start	R/W, COH
	Bit 1	ap_done	R, COR
	Bit 2	ap_idle	R
	Bit 3	ap_ready	R
	Bit 7	auto_restart	R/W
0x04		Global Interrupt Enable	
0x08		Interrupt Enable	
0x0c		Interrupt Status	
0x10		ap_return[31:0]	
0x20 - 0x3f		Argument a: 5x 32 Bit	

Algorithmen-Synthese

Algorithmen-Synthese

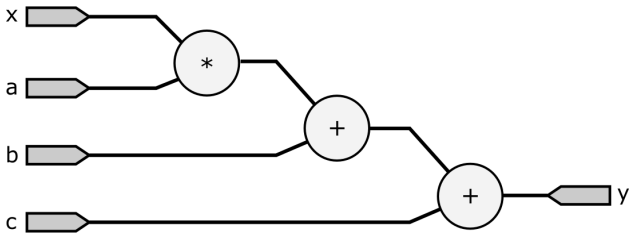
- Eine digitale Schaltung besteht in der Regel aus einem ***Datenpfad*** (eng.: *data path*), welcher die Datenverarbeitung implementiert, und
- einem ***Steuerwerk*** (engl.: *control*), wobei das Steuerwerk die Ausführung des Datenpfads steuert.

- Dieses Modell liegt auch der HLS zu Grunde, so dass durch die HLS eine Prozessoreinheit synthetisiert wird, welche im Unterschied zu einem Mikroprozessor eine ganz spezifische Aufgabe löst.
- Zusätzlich zum Steuerwerk und Datenpfad kommen noch entsprechende Speicherblöcke hinzu, auch diese werden durch die HLS erzeugt.

Datenflussgraphen

- Die Algorithmus-Synthese beginnt mit der Extraktion eines ***Datenflussgraphen*** (engl.: *Data Flow Graph, DFG*) aus dem Quellcode, mit welchem die Datenabhängigkeiten analysiert werden können.
- Ein *Knoten* des DFG repräsentiert eine Operation im Code und die *Kanten* des DFG stellen die Datenabhängigkeiten dar.

```
int ex1(char x, char a, char b, char c) {
  char y;
  y = x * a + b + c;
  return y;
}
```



(1) Scheduling

- Die Algorithmus-Synthese besteht im Wesentlichen aus drei Schritten:

(1) **Scheduling**: Das Scheduling (dt.: Ablaufplanung) bestimmt, welche Operationen aus dem DFG in welchem Taktschritt ausgeführt werden.
  - Beim Scheduling wird auch festgelegt, durch welche Komponenten aus der Ziel-Technologie die Operationen ausgeführt werden sollen. Dies wird auch als *Resource Allocation* bezeichnet.
  - Für eine Operation kann es dabei je nach Ziel-Technologie mehrere Komponenten geben, welche die Operation implementieren können, sich aber im Ressourcenverbrauch und in der Ausführungszeit unterscheiden können.
  - Das Scheduling versucht, einen Ablauf so zu planen, dass die Vorgaben des Benutzers erreicht werden können (im Wesentlichen Ressourcenverbrauch, Latenzzeit, Taktperiode).

(2) Binding

(2) **Binding**: Beim Binding (dt.: Bindung) werden die Operationen aus dem Ablaufplan den konkreten Komponenten zugewiesen. Ferner werden auch Speicherelemente (z.B. Arrays) entsprechenden Ressourcen zugewiesen.

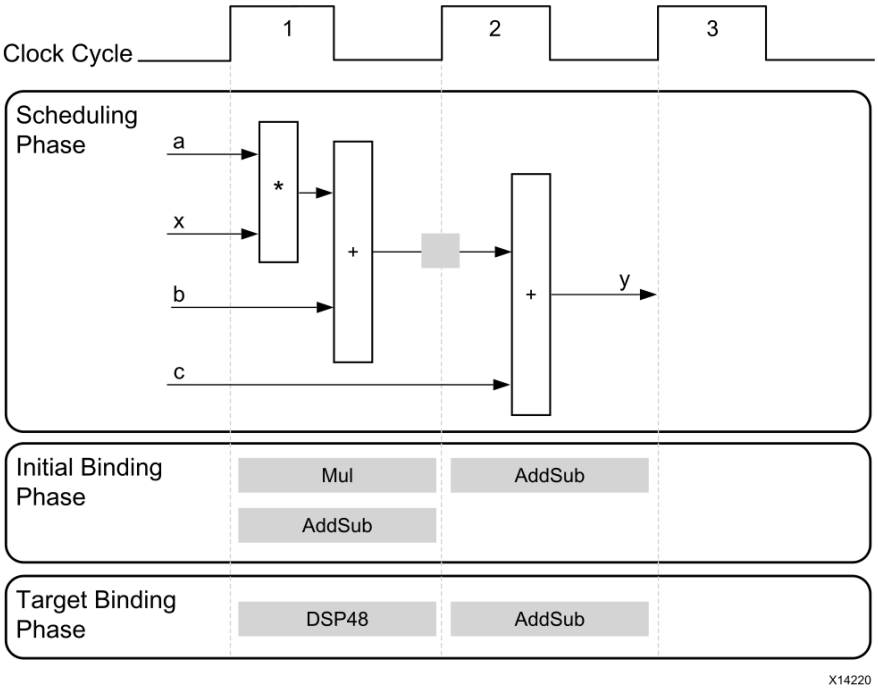
- Ein Ziel ist es dabei, eine Mehrfachnutzung der Ressourcen oder Komponenten zu erreichen, um Ressourcen einsparen zu können. Operationen des gleichen Typs (z.B. Addition/Subtraktion) können beispielsweise durch die gleiche Komponente ausgeführt werden, wenn diese Operationen in unterschiedlichen Taktzyklen ausgeführt werden.
- Ferner können beispielsweise auch Operationen kombiniert werden: Eine Multiplikation mit nachfolgender Addition kann möglicherweise in einem DSP-Block ausgeführt werden.
- Für die Mehrfachnutzung von Komponenten kann es dann erforderlich sein, entsprechende Multiplexer einzuführen.

(3) Extraktion der Steuerlogik

(3) **Extraktion der Steuerlogik**: Ist das Scheduling und Binding abgeschlossen, so wird die benötigte Steuerlogik implementiert.

- Hierzu wird ein Schaltwerk oder Zustandsautomat (engl: Finite State Machine, FSM) generiert, welcher die benötigten Steuersignale in den einzelnen Taktschritten erzeugt.

Beispiel: Scheduling und Binding



Beispiel: Extraktion der Steuerlogik (I)

- Periode T = 9ns

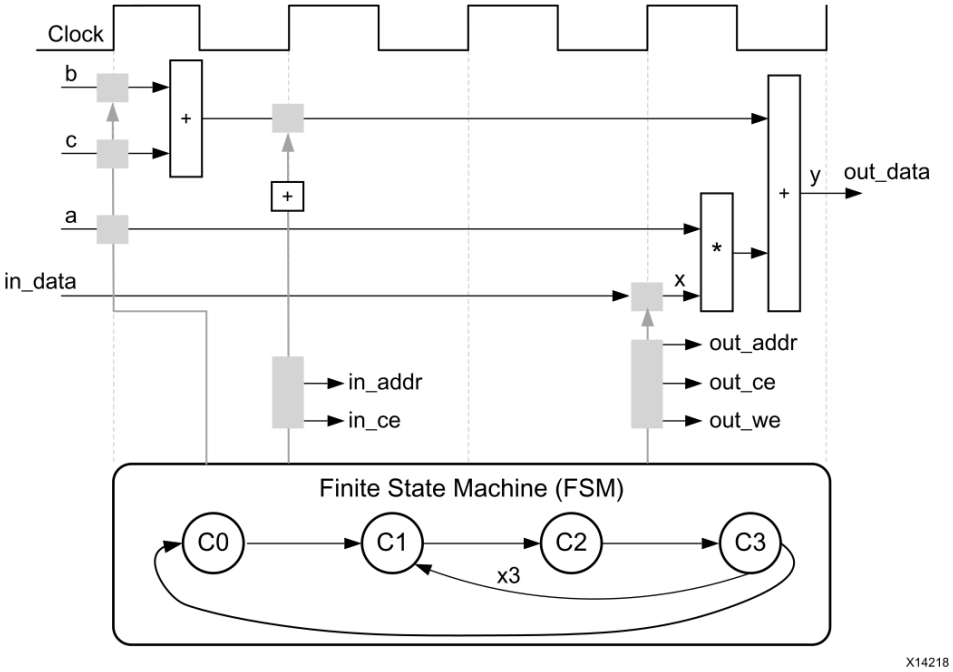
	Operation\Control Step	C0	C1
1	a_read(read)		
2	x_read(read)		
3	tmp(*)		
4	c_read(read)		
5	b_read(read)		
6	tmp1(+)		
7	y(+)		

- Periode T = 5ns

	Operation\Control Step	C0	C1	C2	C3
1	a_read(read)				
2	x_read(read)				
3	tmp(*)				
4	c_read(read)				
5	b_read(read)				
6	tmp1(+)				
7	y(+)				

Beispiel: Extraktion der Steuerlogik (II)

- Periode T = 5ns



Standard C/C ++ -Datentypen (I)

- Wie aus dem Entwurf mit VHDL bekannt ist, spielt die Wahl des Datentypen eine große Rolle im Hinblick auf Ressourcenverbrauch und Performance.
- Dies sind die folgenden Datentypen (mit Angabe der Größe in Bit, Integer jeweils vorzeichenlos und vorzeichenbehaftet):
  - (unsigned) char (8 Bit)
  - (unsigned) short int (16 Bit)
  - (unsigned) long int (32 Bit)
  - (unsigned) long long int (64 Bit)
  - float (32 Bit)
  - double (64 Bit)

Standard C/C ++ -Datentypen (II)

```
typedef long int in_t;
typedef long long int out_t;

out_t ex3(in_t a, in_t b) {
  return (out_t)a * b;
}
```

in_t	out_t	II	BRAM	DSP	FF	LUT	Rg [%]	D [MHz]
char	short int	1	0	1	0	0	0,00278	100,00
short int	long int	1	0	1	0	0	0,00278	100,00
long int	long long int	6	0	4	6	1	0,01116	16,67
float	float	4	0	3	147	141	0,01091	25,00
double	double	6	0	11	323	205	0,03496	16,67

Integer-Datentypen mit variabler Bitbreite

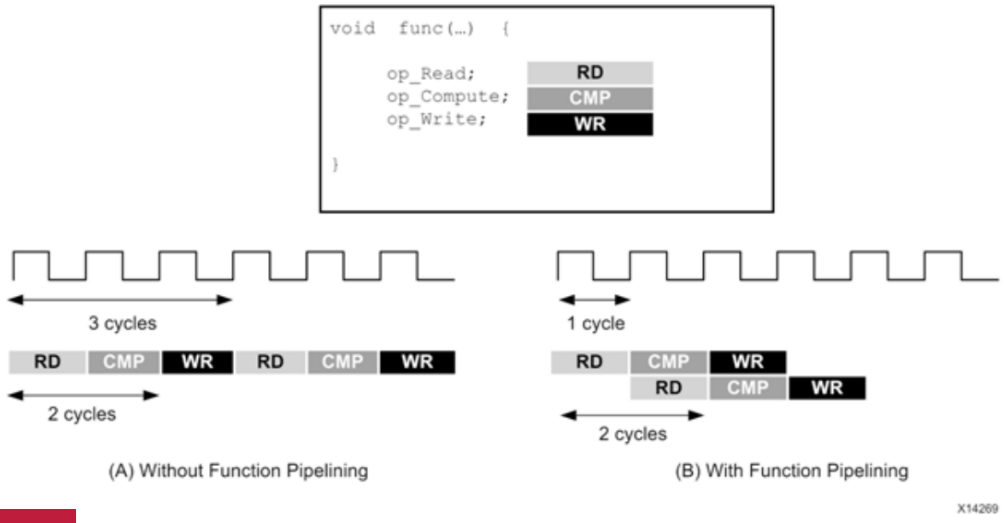
- Eine bessere Kontrolle über die Bitbreite der generierten Hardware kann man durch den Einsatz von speziellen Integer-Datentypen mit variabler Bitbreite (engl.: arbitrary precision data types) erhalten.
- Dies ist vergleichbar mit entsprechenden Datentypen in VHDL, Verilog oder SystemC.
- Um diese verwenden zu können, müssen entsprechende Header-Dateien dazugebunden werden.

```
#include "ap_int.h"

typedef ap_int<4> in_t;
typedef ap_int<8> out_t;

out_t ex3(in_t a, in_t b) {
  return (out_t)a * b;
}
```

in_t	out_t	II	BRAM	DSP	FF	LUT	Rg [%]	D [MHz]
8	16	1	0	1	0	0	0,00278	100,00
16	32	1	0	1	0	0	0,00278	100,00
32	64	6	0	4	6	1	0,01116	16,67
64	128	18	0	16	18	15	0,04473	5,56
128	256	67	0	64	67	154	0,18003	1,49

(1) Pipelining von Funktionen (I)

- Pipelining wird in digitalen Schaltungen eingesetzt, um den Durchsatz zu erhöhen, wie beispielsweise bei leistungsfähigen Mikroprozessoren. Auch die High-Level Synthese setzt Pipelining zur Durchsatzerhöhung ein.



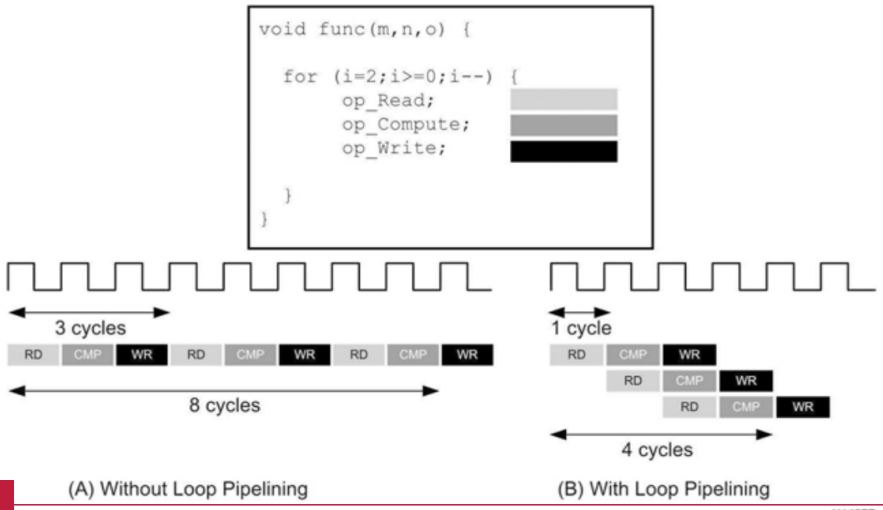
(1) Pipelining von Funktionen (II)

```
typedef ap_int<32> in_t;
typedef ap_int<64> out_t;

out_t ex6(in_t a, in_t b, in_t c, in_t d) {
#pragma HLS PIPELINE

  out_t result;
  result = (out_t)(a + b) * (c + d);
  return result;
}
```

PIPELINE	T	L	II	BRAM	DSP	FF	LUT	Rg [%]	D [MHz]
nein	20	0	1	0	4	0	69	0,01194	50,00
nein	10	6	7	0	4	73	68	0,01237	14,29
ja	10	6	1	0	4	73	66	0,01234	100,00
ja	5	7	1	0	4	74	66	0,01235	200,00
ja	3	7	1	0	4	74	66	0,01235	333,33

(2) Pipelining von Schleifen (I)

- Neben dem Pipelining von ganzen Funktionen, kann auch die Ausführung von einzelnen Schleifen in einer Funktion mit Pipelining versehen werden.
- Hierbei muss betont werden, dass eine Schleife nicht entrollt werden muss, um sie im Pipelining zu betreiben.



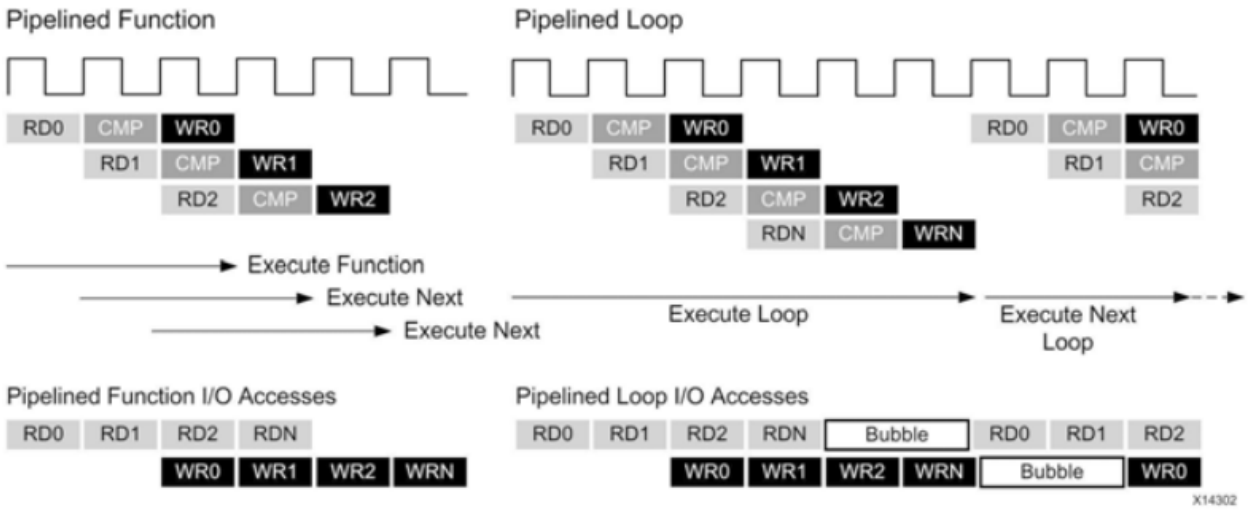
(A) Without Loop Pipelining (B) With Loop Pipelining

(2) Pipelining von Schleifen (II)

```
typedef ap_int<32> in_t;
typedef ap_int<64> out_t;

out_t ex8(in_t a[10], in_t b) {
  out_t result;
  L1: for(int i=0; i<10; i++){
  #pragma HLS PIPELINE
    result = result + a[i] * b;
  }
  return result;
}
```

PIPELINE	T	L	II	BRAM	DSP	FF	LUT	Rg [%]	D [MHz]
nein	10	91	92	0	4	242	78	0,01350	1,09
ja	10	19	20	0	4	241	77	0,01348	5,00

(2) Pipelining von Schleifen (III)

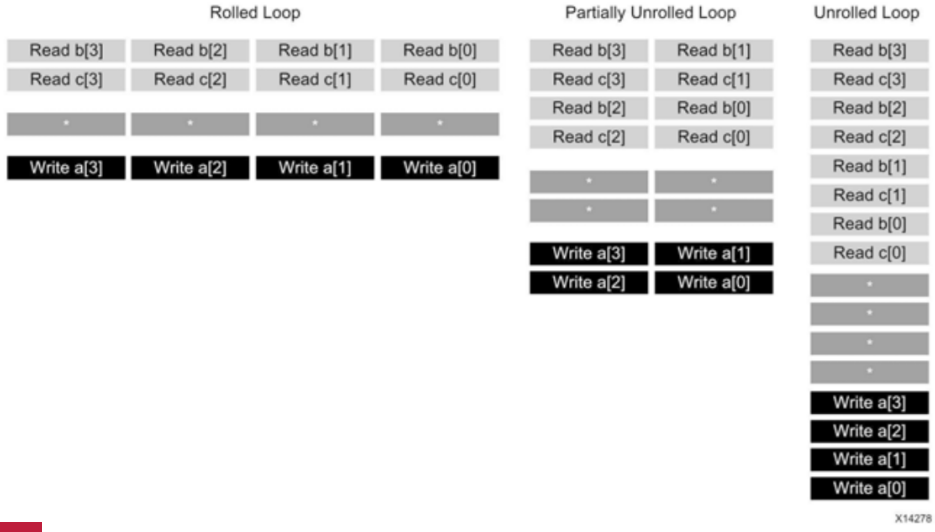


(3) Optimierung von Schleifen durch Entrollen (I)

- Durch das Entrollen einer Schleife kann ebenfalls die Performance erhöht werden.
- Im Unterschied zum Pipelining einer Schleife wird die Ausführung des Schleifenkörpers nicht über die Iterationen verschränkt, sondern der Schleifenkörper wird in der Hardwareimplementierung mehrfach instanziert, so dass eine Parallelverarbeitung von mehreren Iterationen des gesamten Schleifenkörpers erreicht wird.
- Während das Pipelining einer Schleife in der Regel zu einem moderaten Anstieg des Ressourcenverbrauchs führt, kann durch das Entrollen einer Schleife der Ressourcenverbrauch sehr stark ansteigen, insbesondere, wenn die Schleife vollständig entrollt wird oder wenn beispielsweise verschachtelte Schleifen entrollt werden.

(3) Optimierung von Schleifen durch Entrollen (II)

```
void  top(...) {

 ...
 for_mult:for (i=3;i>0;i--)  {
       a[i] = b[i] * c[i];
  }
 ...
}
```



(3) Optimierung von Schleifen durch Entrollen (III)

```
typedef ap_int<16> in_t;
void ex11(in_t a[10], in_t r[10]) {
const static in_t b[10] = {1,2,3,4,5,6,7,8,9,10};

  L1: for(int i=0; i<10; i++){
  #pragma HLS UNROLL factor=2
    r[i] = a[i] + b[i];
  }
}
```

```
void ex11(in_t a[10], in_t r[10]) {
const static in_t b[10] = {1,2,3,4,5,6,7,8,9,10};

  L1: for(int i=0; i<10; i += 2){
    r[i] = a[i] + b[i];
    r[i+1] = a[i+1] + b[i+1];
  }
}
```

UNROLL	T	L	II	BRAM	DSP	FF	LUT	Rg [%]	D [MHz]
nein	10	11	12	0	0	6	155	0,00190	8,33
2	10	6	7	0	0	6	231	0,00281	14,29
5	10	3	4	0	0	6	191	0,00233	25,00
10	10	0	1	0	0	0	160	0,00192	100,00

HLS-Entwurfsablauf (DEMO-Video in Stud.IP)

Literatur

- Frank Kesel, "FPGA Hardware-Entwurf. Schaltungs- und System-Design mit VHDL und C/C++", Walter de Gruyter GmbH, 2018
- Jürgen Reichardt, Bernd Schwarz, "VHDL-Simulation und -Synthese", Walter de Gruyter GmbH, 2020
- Peter J. Ashenden, "The Designer's Guide to VHDL", Morgan Kaufmann, 2010.
- Janick Bergeron, "Writing Testbenches: Functional Verification of HDL Models", Springer Science & Business Media, 2012.